

CSS Flexbox Align-Self

Complete Guide: From Basics to Advanced

Contents

1	Introduction	2
1.1	What is align-self?	2
1.2	align-self vs. align-items	2
1.3	Why Use align-self?	2
1.4	Values Available	3
2	Understanding align-self	4
2.1	How align-self Works	4
2.2	Cascade of Alignment	4
2.3	Common Use Cases	4
3	All align-self Values	6
3.1	auto (Default)	6
3.1.1	Syntax	6
3.1.2	Characteristics	6
3.1.3	Examples	6
3.2	stretch	7
3.2.1	Syntax	7
3.2.2	Characteristics	7
3.2.3	Examples	7
3.3	flex-start	8
3.3.1	Syntax	8
3.3.2	Characteristics	9
3.3.3	Examples	9
3.4	flex-end	10
3.4.1	Syntax	10
3.4.2	Characteristics	10
3.4.3	Examples	10
3.5	center	12
3.5.1	Syntax	12
3.5.2	Characteristics	12
3.5.3	Examples	12
3.6	baseline	14
3.6.1	Syntax	14
3.6.2	Characteristics	14
3.6.3	Examples	14

4	Practical Applications	15
4.1	Application 1: Navigation with Logo	15
4.2	Application 2: Card Layout with Hero	16
4.3	Application 3: Form with Mixed Elements	16
4.4	Application 4: Dashboard with Varied Heights	17
5	Advanced Techniques	19
5.1	Combining Multiple Alignments	19
5.2	Responsive align-self	19
5.3	With Nested Flexbox	20
6	Troubleshooting	21
6.1	Issue 1: align-self Not Working	21
6.2	Issue 2: Conflicting Alignments	21
6.3	Issue 3: Wrong Direction	22
7	Best Practices	23
7.1	When to Use align-self	23
7.2	When NOT to Use align-self	23
7.3	Common Patterns	23
7.4	Accessibility Tips	24
8	Summary: Key Takeaways	25
8.1	The Six Values	25
8.2	Key Rules	25
8.3	Quick Reference	25
8.4	Hierarchy of Alignment	26
8.5	Common Use Cases	26
8.6	Comparison: Alignment Properties	27

Chapter 1

Introduction

1.1 What is align-self?

The `align-self` property allows individual flex items to override the container's `align-items` setting. It aligns a single item on the cross axis, independently from other items.

Key Point: `align-self` is the item-level override for `align-items`. It affects only that specific item, not the entire flex container.

1.2 align-self vs. align-items

Understanding the hierarchy is important:

align-items (Container)	align-self (Item)
Applies to all items	Applies to one item
Set on flex container	Set on flex item
Default for all items	Overrides align-items
Lower priority	Higher priority

1.3 Why Use align-self?

- Break alignment uniformity when needed
- Highlight or differentiate specific items
- Create complex layouts without nested containers
- Handle special cases elegantly
- Fine-tune item positioning
- Maintain semantic HTML structure
- Reduce CSS complexity

1.4 Values Available

`align-self` accepts the same values as `align-items` plus one additional value:

- `auto` - Use container's `align-items` (default)
- `stretch` - Stretch item to fill container
- `flex-start` - Align to start of cross axis
- `flex-end` - Align to end of cross axis
- `center` - Center on cross axis
- `baseline` - Align to baseline

Chapter 2

Understanding align-self

2.1 How align-self Works

```
/* Container sets default alignment */
.flex-container {
  display: flex;
  align-items: center; /* All items centered by default */
}

/* Individual items can override */
.flex-item {
  align-self: flex-start; /* This item goes to top */
}

.flex-item-bottom {
  align-self: flex-end; /* This item goes to bottom */
}

.flex-item-stretch {
  align-self: stretch; /* This item stretches */
}

/* Other items still use align-items: center */
```

2.2 Cascade of Alignment

1. If align-self is set → Use align-self
2. Else if align-items is set → Use align-items
3. Else → Default is stretch

2.3 Common Use Cases

- Aligning a specific item differently
- Creating highlight or hero items

- Handling items with different heights
- Emphasizing certain content
- Building complex layouts
- Responsive design adjustments

Chapter 3

All align-self Values

3.1 auto (Default)

Uses the container's `align-items` value.

3.1.1 Syntax

```
.flex-item {  
    align-self: auto; /* Default */  
}
```

3.1.2 Characteristics

- Inherits container alignment
- No override applied
- Item follows group alignment
- Default state

3.1.3 Examples

Example 1: Default Inheritance

```
<div class="container">  
    <div class="item">Item 1</div>  
    <div class="item">Item 2</div>  
    <div class="item">Item 3</div>  
</div>  
  
.container {  
    display: flex;  
    align-items: center; /* All items centered */  
    height: 200px;  
    gap: 20px;  
}  
  
.item {
```



```
    align-self: auto; /* Follows container alignment */
    background: #3498db;
    padding: 20px;
}

/* All items centered because align-self: auto */
```

3.2 stretch

Item stretches to fill the container on the cross axis.

3.2.1 Syntax

```
.flex-item {
    align-self: stretch;
}
```

3.2.2 Characteristics

- Item grows to fill available space
- Overrides container alignment
- Creates variable heights
- Useful for full-height content

3.2.3 Examples

Example 1: Mixed Heights

```
<div class="container">
  <div class="item">Short</div>
  <div class="item featured">Featured</div>
  <div class="item">Short</div>
</div>

.container {
    display: flex;
    align-items: center;
    height: 300px;
    gap: 20px;
}

.item {
    background: white;
    padding: 20px;
    border-radius: 8px;
}

.item.featured {
```

```
    align-self: stretch; /* This item full height */
    background: #3498db;
    color: white;
}

/* Middle item stretches, others centered */
```

Example 2: Sidebar with Hero

```
<div class="layout">
  <aside class="sidebar">Sidebar</aside>
  <article class="hero">Featured</article>
  <section class="content">Content</section>
</div>

.layout {
  display: flex;
  align-items: center;
  height: 400px;
  gap: 20px;
}

.sidebar {
  width: 200px;
  background: #ecf0f1;
  padding: 20px;
}

.hero {
  align-self: stretch; /* Full height */
  flex: 1;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  padding: 40px;
  border-radius: 8px;
}

.content {
  width: 200px;
  background: #ecf0f1;
  padding: 20px;
}

/* Hero section stretches to full height */
```

3.3 flex-start

Item aligns to the start of the cross axis.

3.3.1 Syntax

```
.flex-item {
```

```
    align-self: flex-start;
}
```

3.3.2 Characteristics

- Item at top (for row) or left (for column)
- Overrides container alignment
- Item maintains natural size
- Space below/right item

3.3.3 Examples

Example 1: Aligned Item in Centered Group

```
<div class="container">
  <div class="item">Normal 1</div>
  <div class="item top">At Top</div>
  <div class="item">Normal 2</div>
</div>

.container {
  display: flex;
  align-items: center;
  height: 200px;
  gap: 20px;
}

.item {
  background: #3498db;
  padding: 20px;
  color: white;
}

.item.top {
  align-self: flex-start; /* Goes to top */
}

/* One item at top, others centered */
```

Example 2: Icon and Content

```
<div class="card">
  <div class="icon "></div>
  <div class="content">
    <h3>Alert</h3>
    <p>This is an alert message
    that spans multiple lines
    and is quite long.</p>
  </div>
</div>
```

```
.card {
  display: flex;
  align-items: center;
  gap: 15px;
  padding: 20px;
  background: white;
  border-radius: 8px;
}

.icon {
  font-size: 32px;
  align-self: flex-start; /* Icon at top */
  margin-top: 5px;
}

.content h3 {
  margin-top: 0;
}

/* Icon aligns to top of content */
```

3.4 flex-end

Item aligns to the end of the cross axis.

3.4.1 Syntax

```
.flex-item {
  align-self: flex-end;
}
```

3.4.2 Characteristics

- Item at bottom (for row) or right (for column)
- Overrides container alignment
- Item maintains natural size
- Space above/left item

3.4.3 Examples

Example 1: Badge on Item

```
<div class="product-row">
  <div class="product">Product 1</div>
  <div class="product">
    <span>Product 2</span>
    <span class="badge">New</span>
  </div>
```

```
<div class="product">Product 3</div>
</div>

.product-row {
  display: flex;
  align-items: center;
  height: 150px;
  gap: 20px;
}

.product {
  flex: 1;
  background: white;
  padding: 20px;
  border-radius: 8px;
  position: relative;
}

.badge {
  align-self: flex-end;
  position: absolute;
  top: 10px;
  right: 10px;
  background: #e74c3c;
  color: white;
  padding: 4px 8px;
  border-radius: 12px;
}

/* Badge at bottom of product */
```

Example 2: Footer Actions

```
<div class="modal-content">
  <h2>Modal Title</h2>
  <p>Modal content here</p>
  <div class="actions">
    <button class="cancel">Cancel</button>
    <button class="confirm">Confirm</button>
  </div>
</div>

.modal-content {
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.actions {
  display: flex;
  gap: 10px;
  align-self: flex-end; /* Actions at bottom-right */
}

.actions button {
```

```
padding: 8px 16px;
border: none;
border-radius: 4px;
cursor: pointer;
}

/* Buttons aligned to bottom */
```

3.5 center

Item is centered on the cross axis.

3.5.1 Syntax

```
.flex-item {
  align-self: center;
}
```

3.5.2 Characteristics

- Item centered on cross axis
- Overrides other alignments
- Creates emphasis
- Professional appearance

3.5.3 Examples

Example 1: Hero in Mixed Layout

```
<div class="section">
  <div class="text">Short text</div>
  <div class="featured">Featured Item</div>
  <div class="text">Another text</div>
</div>

.section {
  display: flex;
  align-items: flex-start;
  height: 300px;
  gap: 20px;
}

.text {
  flex: 1;
  background: #ecf0f1;
  padding: 20px;
}
```

```
.featured {
  flex: 1;
  align-self: center; /* Centered while others at top */
  background: #3498db;
  color: white;
  padding: 30px;
  border-radius: 8px;
  text-align: center;
}

/* Featured item centered vertically */
```

Example 2: Icon Highlight

```
<div class="features">
  <div class="feature">
    <span class="icon ">1</span>
    <p>Step one text</p>
  </div>
  <div class="feature highlight">
    <span class="icon "></span>
    <p>Highlighted step</p>
  </div>
  <div class="feature">
    <span class="icon ">3</span>
    <p>Step three text</p>
  </div>
</div>

.features {
  display: flex;
  align-items: flex-start;
  gap: 20px;
}

.feature {
  flex: 1;
}

.feature.highlight {
  align-self: center; /* Highlighted centered */
}

.icon {
  font-size: 36px;
  display: block;
  margin-bottom: 10px;
}

/* Middle feature centered */
```

3.6 baseline

Item aligns to text baseline.

3.6.1 Syntax

```
.flex-item {  
    align-self: baseline;  
}
```

3.6.2 Characteristics

- Item aligned to text baseline
- Useful for mixed font sizes
- Natural text alignment
- Less commonly used

3.6.3 Examples

Example 1: Mixed Text Sizes

```
<div class="heading">  
  <h1>35</h1>  
  <span class="small">views</span>  
</div>  
  
.heading {  
  display: flex;  
  align-items: flex-start;  
  gap: 10px;  
}  
  
.heading h1 {  
  margin: 0;  
  font-size: 48px;  
}  
  
.small {  
  align-self: baseline; /* Align to number baseline */  
  font-size: 14px;  
  color: #666;  
}  
  
/* Text aligned to baseline of large number */
```


Chapter 4

Practical Applications

4.1 Application 1: Navigation with Logo

```
<nav class="navbar">
  
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Contact</a>
  <button class="cta">Sign Up</button>
</nav>

.navbar {
  display: flex;
  align-items: center;
  padding: 15px 30px;
  background: #2c3e50;
  gap: 20px;
}

.navbar a {
  color: white;
  text-decoration: none;
  flex: 1;
}

.logo {
  width: 40px;
  height: 40px;
  align-self: flex-start; /* Logo at top */
}

.cta {
  background: #3498db;
  color: white;
  border: none;
  padding: 8px 16px;
  border-radius: 4px;
  cursor: pointer;
  align-self: flex-end; /* Button at bottom */
}
```

```
}  
  
/* Mixed alignment in single row */
```

4.2 Application 2: Card Layout with Hero

```
<div class="card-grid">  
  <div class="card">  
    <h3>Card 1</h3>  
    <p>Content</p>  
  </div>  
  <div class="card featured">  
    <h2>Featured</h2>  
    <p>Important content here</p>  
  </div>  
  <div class="card">  
    <h3>Card 3</h3>  
    <p>Content</p>  
  </div>  
</div>  
  
.card-grid {  
  display: flex;  
  align-items: stretch;  
  gap: 20px;  
  min-height: 300px;  
}  
  
.card {  
  flex: 1;  
  background: white;  
  padding: 20px;  
  border-radius: 8px;  
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);  
}  
  
.card.featured {  
  align-self: center; /* Featured centered */  
  transform: scale(1.05);  
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.2);  
}  
  
/* Featured card stands out */
```

4.3 Application 3: Form with Mixed Elements

```
<form class="form-layout">  
  <label>Name:</label>  
  <input type="text" placeholder="Enter name">
```

```
<label>Email:</label>
<input type="email" placeholder="Enter email">

<label></label>
<button type="submit">Submit</button>
</form>

.form-layout {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 15px;
}

.form-layout label {
  flex: 0 0 100px;
  text-align: right;
}

.form-layout input {
  flex: 1;
  padding: 8px;
  border: 1px solid #ddd;
}

.form-layout button {
  flex: 1;
  padding: 10px;
  background: #27ae60;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  align-self: flex-end; /* Button at bottom */
}

/* Button positioned separately */
```

4.4 Application 4: Dashboard with Varied Heights

```
<div class="dashboard">
  <div class="widget small">Small</div>
  <div class="widget large">Large</div>
  <div class="widget medium">Medium</div>
</div>

.dashboard {
  display: flex;
  align-items: flex-start;
  gap: 20px;
  min-height: 400px;
}
```

```
.widget {  
  flex: 1;  
  background: white;  
  padding: 20px;  
  border-radius: 8px;  
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);  
}  
  
.widget.large {  
  align-self: stretch; /* Full height */  
}  
  
.widget.medium {  
  align-self: center; /* Centered */  
}  
  
/* Different heights for emphasis */
```

Chapter 5

Advanced Techniques

5.1 Combining Multiple Alignments

```
.container {  
    display: flex;  
    align-items: center;  
    height: 300px;  
}  
  
.item-top {  
    align-self: flex-start;  
}  
  
.item-center {  
    align-self: center;  
}  
  
.item-bottom {  
    align-self: flex-end;  
}  
  
.item-stretch {  
    align-self: stretch;  
}  
  
/* All alignments in one container */
```

5.2 Responsive align-self

```
.item {  
    align-self: flex-start;  
}  
  
/* Change on larger screens */  
@media (min-width: 768px) {  
    .item {  
        align-self: center;  
    }  
}
```

```
    }  
}  
  
@media (min-width: 1200px) {  
    .item {  
        align-self: stretch;  
    }  
}
```

5.3 With Nested Flexbox

```
.outer {  
    display: flex;  
    align-items: center;  
}  
  
.inner {  
    display: flex;  
    align-items: flex-start;  
}  
  
.inner .item {  
    align-self: center; /* Overrides parent alignment */  
}  
  
/* Multiple levels of alignment control */
```

Chapter 6

Troubleshooting

6.1 Issue 1: align-self Not Working

Problem: align-self has no visible effect.

Solution: Ensure parent has display: flex and height constraint.

```
/* WRONG: Parent has no height */
.container {
  display: flex;
}

.item {
  align-self: flex-end; /* No visible effect */
}

/* CORRECT: Add height to parent */
.container {
  display: flex;
  height: 200px; /* Now it works */
}
```

6.2 Issue 2: Conflicting Alignments

Problem: align-self conflicts with align-items.

Solution: Remember align-self has higher priority.

```
.container {
  display: flex;
  align-items: center; /* Default for all */
}

.special {
  align-self: flex-start; /* Overrides for this item */
}

/* align-self wins */
```

6.3 Issue 3: Wrong Direction

Problem: Item aligns in unexpected direction.

Solution: Check `flex-direction` - it determines cross axis.

```
/* Row: align-self affects vertical */
.horizontal {
    display: flex;
    flex-direction: row;
    height: 200px;
}

.horizontal .item {
    align-self: center; /* Vertical centering */
}

/* Column: align-self affects horizontal */
.vertical {
    display: flex;
    flex-direction: column;
    width: 200px;
}

.vertical .item {
    align-self: center; /* Horizontal centering */
}
```


Chapter 7

Best Practices

7.1 When to Use align-self

- Override alignment for specific items
- Create emphasis or highlights
- Handle special cases
- Vary item positioning
- Create complex asymmetric layouts
- Responsive design adjustments

7.2 When NOT to Use align-self

- For all items (use align-items instead)
- When semantic meaning changes
- For layout hacks
- When it makes code confusing

7.3 Common Patterns

```
/* Highlight one item */
.featured {
  align-self: stretch;
}

/* Icon at top, text centered */
.icon-text {
  align-self: flex-start;
}

/* Button at bottom of form */
```

```
.submit-button {
  align-self: flex-end;
}

/* Special item centered among others */
.special {
  align-self: center;
}

/* Mixed alignment for emphasis */
.container .item:nth-child(2) {
  align-self: flex-start;
}

.container .item:nth-child(3) {
  align-self: center;
}
```

7.4 Accessibility Tips

- Don't use align-self to hide content
- Maintain logical reading order
- Ensure alignment serves a purpose
- Test keyboard navigation
- Consider screen reader experience

Chapter 8

Summary: Key Takeaways

8.1 The Six Values

Value	Effect
auto	Uses container alignment
stretch	Stretches to fill space
flex-start	Aligns to start
flex-end	Aligns to end
center	Aligns centered
baseline	Aligns to baseline

8.2 Key Rules

1. Works only on flex items
2. Overrides container `align-items`
3. Applies to single item only
4. Works on cross axis
5. Requires parent `display: flex`
6. Best for exceptions, not default behavior
7. Responsive with media queries
8. Can be combined with other properties
9. Higher priority than `align-items`
10. Use sparingly for clarity

8.3 Quick Reference

```
/* Reset to default */
.item {
    align-self: auto;
}

/* Item at top */
.item-top {
    align-self: flex-start;
}

/* Item centered */
.item-center {
    align-self: center;
}

/* Item at bottom */
.item-bottom {
    align-self: flex-end;
}

/* Item full height */
.item-stretch {
    align-self: stretch;
}

/* Text baseline */
.item-baseline {
    align-self: baseline;
}

/* Override for special item */
.special {
    align-self: flex-start; /* While others centered */
}
```

8.4 Hierarchy of Alignment

1. `align-self` (highest priority)
2. `align-items` (container default)
3. Browser default: `stretch`

8.5 Common Use Cases

- Make one item different from group
- Highlight featured content
- Create asymmetric layouts
- Handle special items

- Emphasize important elements
- Handle responsive adjustments
- Break visual uniformity intentionally

8.6 Comparison: Alignment Properties

Property	Level	Scope	Axis
align-items	Container	All items	Cross
align-self	Item	Single item	Cross
justify-content	Container	Main axis	Main